

Project Drafting

Describes what frontend wants

Backend to do:

GET: give data (just reading)

POST: here's data, do things

Users:

POST:

register: {userID, password} - backend queries database - ensure db makes userIDs unique: true
prevents race conditions

backend will return error if taken, otherwise backend hashes pass w/ bcrypt, backend sends

user + hashed-pass to db, db inserts, returns success, backend returns success to frontend → tell user success

↳ Note: Make backend insert with unique indexing - db will reject/accept (1 step) backend can just interpret

via try-catch block - continuing if user not taken, returning error 409 to user (conflict w/ data - user taken)

/login: {userID, password} - backend asks db to look up user & return its hashed pass, backend hashes input

password and compares if input-hash == database-hash, return success, otherwise throw error to frontend

HTTP status codes:

2XX - Informational, request received, processing continues

3XX - Success, request was successfully processed

3XX - Redirection, further action needed to complete request

4XX - Client errors, request cannot be fulfilled.

5XX - Server errors, server failed to fulfill a valid request.

Project:

POST:

/create-project {userID, projectID, description}, backend queries db to insert projectID (see /register), insert to db:

upon success → db adds projectID to list of projects under a user & add description to proj.

upon failure → backend propagates error 409 to frontend (projectID taken)

/join-project {userID, projectID}, backend queries db to 1) see if projectID exists → if not, throw error for

invalid ID, and 2) check if projectID matches any projects attached to user in db → if so, throw

error saying user is already part of project

GET:

/get-user-project {userID} also can be retrieved from session, backend queries db for projects attached to user,

either 1) return array/list of projects by ID or 2) if empty, throw "error" saying user is not a part of any project
more like notify, not really error

/get-project-info {projectID} backend queries db, db returns project's description (and name - optional), current

members (maybe the creator of project, see /create-project)

Which hardware set would you like?

Select one ▼

Which project would you like to check out under?

Select one ▼

Hardware:

POST:

user must choose which project they are working under

/check-out {projectID, HWset, quantity} - backend queries db, db checks HWset exist, throw error if not. then checks if quantity < current amount

w/ db, if yes then decrement from available stock & add to project, if no → throw error 1) invalid character if !number or 2) trying to check out more than is available

/check-in {projectID, HWset, quantity} - db checks project has > quantity HWsets → if no, throw error (project doesn't have that many HWsets)

if yes, decrement amount from project allocation & add to available stock

GET:

/get-hw-info - lists all HWsets, their respective capacity and availability → backend queries db, db returns {available, capacity}